KPR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(Autonomous), Coimbatore – 641 407 · Department of CSE

U21CSP12 – JavaScript Frameworks

ASSIGNMENT – II

Detail	
Name	Sathya R V
Roll Number	24CS192
Year / Semester	III / V – B.E. CSE
Academic Year	2026 – 2027
Issue / Submission Date	15-10-2026 / 31-10-2026
GitHub Repository	zylan07/Javascript_Assignment_2: Built 3 Full stack apps for assignment

Questions Answered

Q.No	Task	Application	Stack	CO	Marks
Q1	Student Management System using MERN Stack	EduRoster	MERN	CO3	8
Q2 (Task 1)	Online bookstore application	PageTurn Books	MERN	CO5	16
Q3 (Task 1)	Course registration application for college students	EnrollEase	MEAN	CO4	16

Technologies

MongoDB + Mongoose (database), Express.js + Node.js (REST API), React 18 with Vite (MERN – Q1, Q2), Angular 20 with Reactive Forms (MEAN – Q3). Each app follows Browser → React/Angular → REST/JSON → Express → Mongoose → MongoDB. Complete source code and run steps are in the GitHub repository / zip.

Q1 – EduRoster – Student Management System

Task: *Develop a Student Management System using MERN Stack: React form to enter student details, REST APIs with Express.js and Node.js, student records stored in MongoDB, add / view / update / delete operations.*

Stack: React 18 (Vite) · Node.js + Express.js · MongoDB (sathya_eduroster)

Requirements and Implementation

Requirement	Implementation
React form to enter student details	StudentForm.jsx – controlled inputs for roll no, name, email, phone, department, year, CGPA with validation
REST APIs using Express.js and Node.js	routes/studentRoutes.js + controllers/studentController.js – GET, POST, PUT, DELETE on /api/students
Store student records in MongoDB	models/Student.js – Mongoose schema with validation, unique roll number, timestamps
Add, view, update and delete operations	Add & Update through the form, View in a searchable table, Delete with confirmation

Database

Collection fields: rollNo (String), name (String), email (String), phone (String), department (String), year (Number), cgpa (Number), createdAt / updatedAt (Date).

REST API

Method	Endpoint	Purpose
GET	`/api/students`	List students (query: search, department, year, sort)
GET	`/api/students/:id`	Get one student
POST	`/api/students`	Add a student (409 if roll number exists)
PUT	`/api/students/:id`	Update a student
DELETE	`/api/students/:id`	Delete a student
GET	`/api/students/meta/departments`	Department list for the form

Output

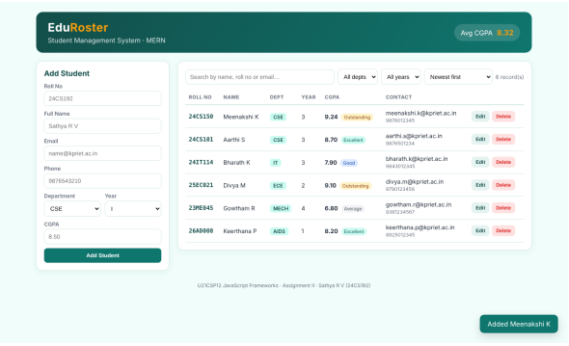


Fig. 1: CREATE – new student added

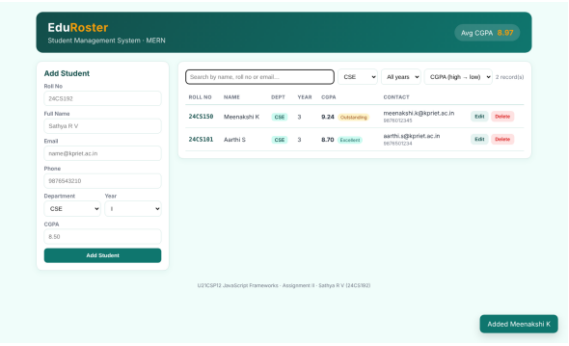


Fig. 2: VIEW – filter by department and sort by CGPA

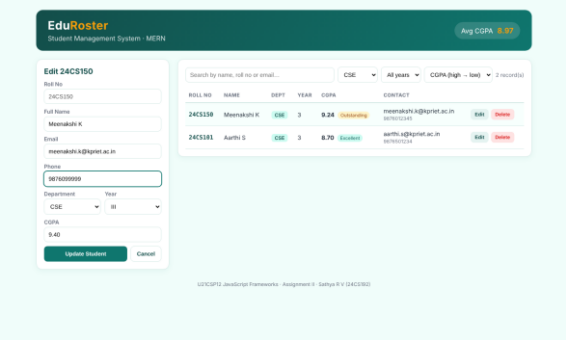


Fig. 3: UPDATE – form in edit mode

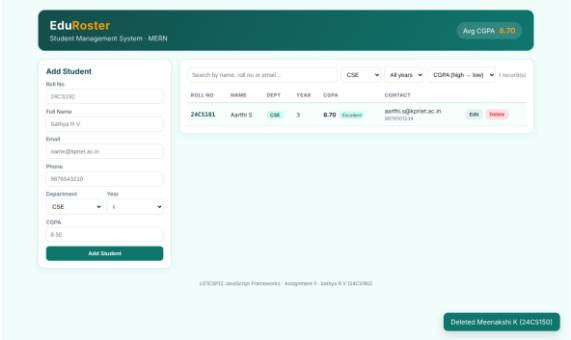


Fig. 4: DELETE – record removed

Result: The application was implemented and all required operations were tested successfully.

Q2 (Task 1) – PageTurn Books – Online Bookstore

Task: *Design an online bookstore application: display books using React components, store book title, author, price and availability in MongoDB, create APIs to retrieve and manage books, implement search by book title or author.*

Stack: React 18 (Vite) · Node.js + Express.js · MongoDB (sathya_pageturn)

Requirements and Implementation

Requirement	Implementation
Display books using React components	BookList → BookCard components render each book as a card with a drawn cover
Store title, author, price and availability in MongoDB	models/Book.js – title, author, price, available (+ stock, genre, year)
Create APIs to retrieve and manage books	GET / POST / PUT / PATCH / DELETE on /api/books + summary endpoint
Search by book title or author	GET /api/books?q=..&searchBy=title|author|all – case-insensitive regex in MongoDB; switch in SearchBar.jsx

Database

Collection fields: title (String), author (String), price (Number), available (Boolean), stock (Number), genre (String), publishedYear (Number), coverColor (String).

REST API

Method	Endpoint	Purpose
GET	`/api/books`	List / search books (query: q, searchBy=title|author|all, genre, available, sort)
GET	`/api/books/:id`	Get one book
POST	`/api/books`	Add a book
PUT	`/api/books/:id`	Edit a book
PATCH	`/api/books/:id/availability`	Toggle In stock / Out of stock
DELETE	`/api/books/:id`	Delete a book
GET	`/api/books/meta/summary`	Totals for the header + genre list

Output

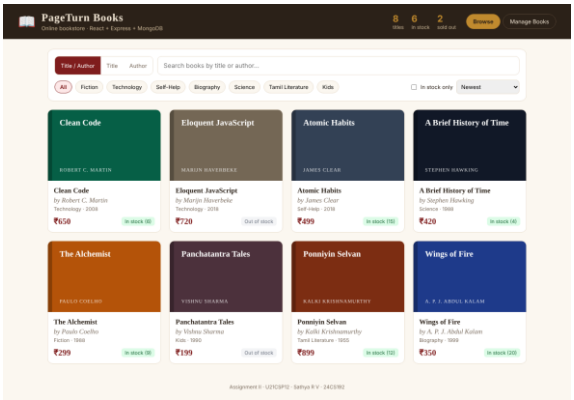


Fig. 5: Bookstore home – books displayed with React components

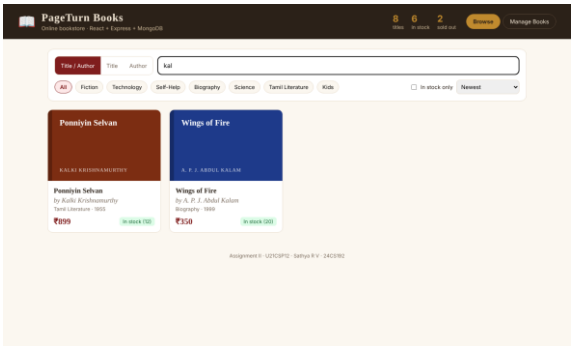


Fig. 6: Search by AUTHOR ("kal" → Kalam, Kalki)

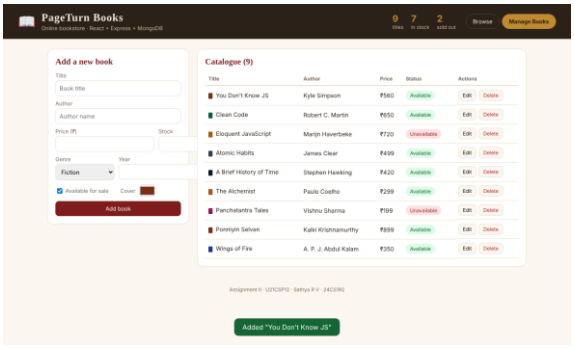


Fig. 7: New book stored in MongoDB and listed

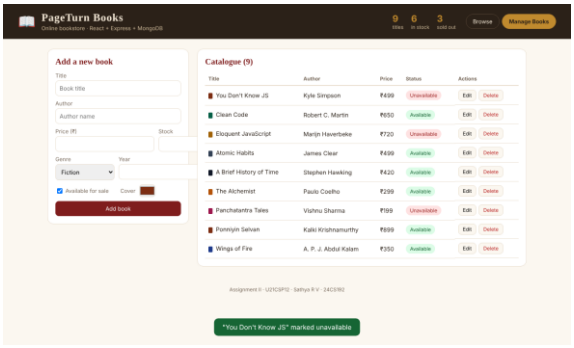


Fig. 8: Availability toggled to Unavailable

Result: The application was implemented and all required operations were tested successfully.

Q3 (Task 1) – EnrollEase – Course Registration (MEAN)

Task: *Create a course registration application for college students: display available courses using Angular, implement course registration and cancellation, store course and student details in MongoDB, validate user inputs using Angular forms.*

Stack: Angular 20 (standalone components, Reactive Forms, signals) · Node.js + Express.js · MongoDB (sathya_enrollease)

Requirements and Implementation

Requirement	Implementation
Display available courses using Angular	CourseList component shows course cards with seats left, category filter and search
Course registration and cancellation	POST /api/registrations (seat, duplicate and credit checks) and PATCH /api/registrations/:id/cancel
Store course and student details in MongoDB	Course, Student and Registration collections linked with ObjectId references
Validate user inputs using Angular forms	Reactive Forms – custom roll-number validator, required, minLength, pattern, email, requiredTrue

Database

Course: code, title, instructor, credits, category, schedule, capacity | Student: rollNo (unique), name, email, department, year | Registration: student (ref), course (ref), status Registered / Cancelled, cancelledAt

REST API

Method	Endpoint	Purpose
GET	`/api/courses`	Courses with enrolled count and seats left
POST	`/api/courses`	Add a course (admin)
DELETE	`/api/courses/:id`	Delete a course (only if nobody registered)
POST	`/api/registrations`	Register a student for selected courses
GET	`/api/registrations/student/:rollNo`	A student's registrations + total credits
PATCH	`/api/registrations/:id/cancel`	Cancel a registration
GET	`/api/registrations`	All registrations (admin)

Output

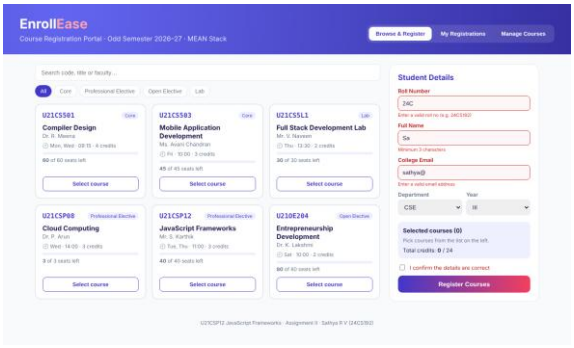


Fig. 9: Angular Reactive Form validation messages

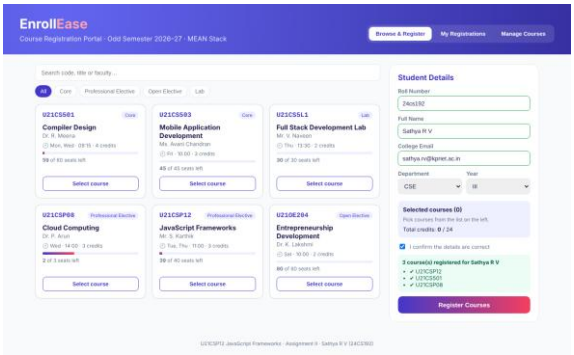


Fig. 10: REGISTRATION – 3 courses registered

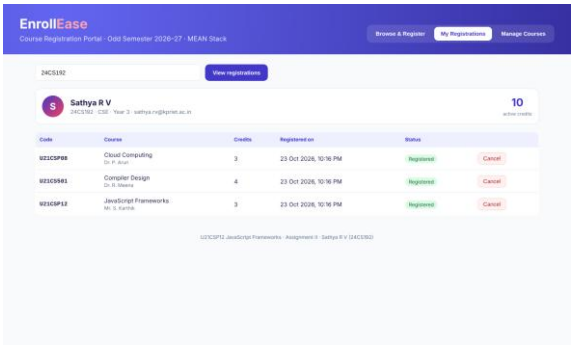


Fig. 11: Student's registrations with total credits

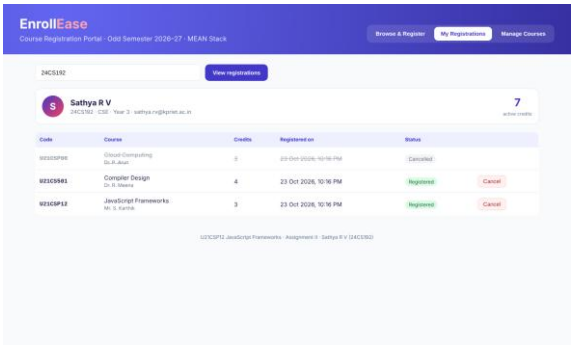


Fig. 12: CANCELLATION – Cloud Computing cancelled

Result: The application was implemented and all required operations were tested successfully.

Conclusion: All three questions were implemented as working MERN/MEAN applications using React Hooks, Angular forms and services, Express REST APIs and MongoDB.